

Stalin: The Game

Robin Langer

1 September 2026

***Disclaimer.** I am not an expert at Unix, or at web app development, so these notes are just as much for myself as for anybody else.*

Introduction

Neil Ghani’s *Containers for Typed Agentic AI* describes an algebra of interfaces. An interface is a pair. The first half is the prompts you may send. The second half gives, for each prompt, the type of response that would answer it. Such interfaces compose, in four different ways. Three of them appear in this game.

Container theory arose out of dependent type theory, and “playing around” with containers is usually done in a language such as Idris or Lean. This is not what we do in these notes. Instead we “fake” dependent types using Unix CLI tools and TypeScript `async` event handlers.

The division of labour between those two is exact. First, the definition. A *morphism* of containers $(P, R) \rightarrow (Q, T)$ is a pair of functions

$$u : P \rightarrow Q \qquad f : (p : P) \rightarrow T (u p) \rightarrow R p.$$

The mapping u *delegates* a task. The mapping f *amalgamates* a response. So a container morphism has two flows, prompts moving forwards and responses moving backwards. Control flows *down* through u . Data flows *back up* through f .

And each of our two ordinary things supplies exactly one of those flows.

CLI tools: The forward map u on shapes is implemented as a Unix CLI tool call.

This works because a command-line tool is already a container. It receives one prompt, its argument vector. It produces a response whose *type depends on which prompt it was*.

TypeScript `async` event handlers: The backwards map f on positions is implemented with TypeScript `async` event handlers.

This works because in a web app control has been inverted. An ordinary program calls a second function and that second function returns an answer. Here the event loop is passed the function as an argument. The function delegates to a second function, and then it hands control back to the event loop before it has an answer of its own.

So the two halves of a morphism are the two directions of control, and they are separate acts. The mapping u runs while your code has control. The function f runs after control has been given away and handed back. In an ordinary function call there would be nothing to tell them apart.

Typed agents. Note that the Claude Code harness is itself a CLI tool, and thus a container. We can simply “plug in” the `claude` binary as our CLI tool and all the machinery developed in this note works exactly the same.

A note on formal verification. The architecture of the game is motivated by Ghani’s theory of container combinators, but the design is not formally verifiable. This was never the point. It is a “fake”, but it is a fake that works.

Contents

1	The game	6
2	The prompt	7
3	Strategy	8
3.1	Why the two costs are not independent	8
3.2	Build them or buy them	8
3.3	Seven plans	8
3.4	A surprise	9
4	The Unix operating system, pipes, and CLI tools	10
4.1	A process	10
4.2	User space and kernel space	10
4.3	<code>read</code> and <code>write</code>	10
4.4	Everything is a file	10
4.5	File descriptors	11
4.6	<code>cat</code>	11
4.7	<code>fork</code> and <code>exec</code>	11
4.8	The shell	12
4.8.1	The environment	12
4.8.2	Globbering	12
4.8.3	What the shell does with a line	12
4.9	Redirection	13
4.10	Pipes	13
4.11	Threads	14
4.12	Exit status, and the shell's control flow	15
4.13	Shell scripts	15
4.14	Three channels, not one	15
4.15	Manual pages, and what is not checked	16
4.16	The dictum	16
4.17	A CLI tool is already a dependent function	17
5	The Claude Code harness	18
6	HTTP and JSON	19
6.1	A request and a response	19
6.2	Deliberately simple	19
6.3	The method says what kind of thing you are doing	20
6.4	JSON	20
6.5	A request and a response, as values	21
7	TypeScript syntax	22
7.1	Basic types	22
7.2	The weird stuff	22
7.3	Type guards	23
7.4	Subtypes	24
7.5	The powerful stuff	24
7.6	<code>async</code> functions	24
7.7	On the level of values	24
8	Structural typing	25

8.1	A tagged union writes down a finite choice	25
9	Conditional types	26
9.1	A function from types to types	26
9.2	Question for Orestis	26
9.3	A function on the level of values	27
9.4	The fake container	27
9.5	Another question for Orestis	27
9.6	Two limits	27
9.7	Question for Orestis	28
9.8	Recovering the name	28
9.9	Mapped types, which make a table total	28
10	Compiling down to JavaScript	30
10.1	Types are checked before the program runs	30
10.2	A cast is a promise, not a check	30
10.3	<code>unknown</code> is what you want at a boundary	31
10.4	Validate, do not cast	31
11	<code>async</code>, <code>await</code>, and the event loop	33
11.1	One flow of control, and it may not stop	33
11.2	How it actually works	34
11.3	Inversion of control	35
11.4	A promise is the pending computation as a value	35
11.5	What <code>await</code> actually does, which is not waiting	35
12	How Deno runs a CLI tool	37
12.1	Deno can call Claude Code	38
13	Delegation	39
14	A container in TypeScript	40
15	Runners	42
15.1	Decoders	42
15.2	Pass-through runners	42
15.3	And this is the morphism	43
16	The tensor combinator	45
16.1	Concurrency is not parallelism	45
16.2	The tensor combinator	45
17	The other three combinators	47
18	Formal verifiability	48
19	How the game is implemented	49
19.1	Five servers, and why they are servers	49
19.2	The game as a composite	49
19.3	What crosses, and what the status code says	49
19.4	A hop is a CLI tool	50
19.5	Where the compiler carries a rule of the game	50
19.6	The one thing that cannot cross	51

19.7	What the reports are worth	51
19.8	And where the protocol is not doing the work	51
19.9	The response flows backwards	52

1 The game

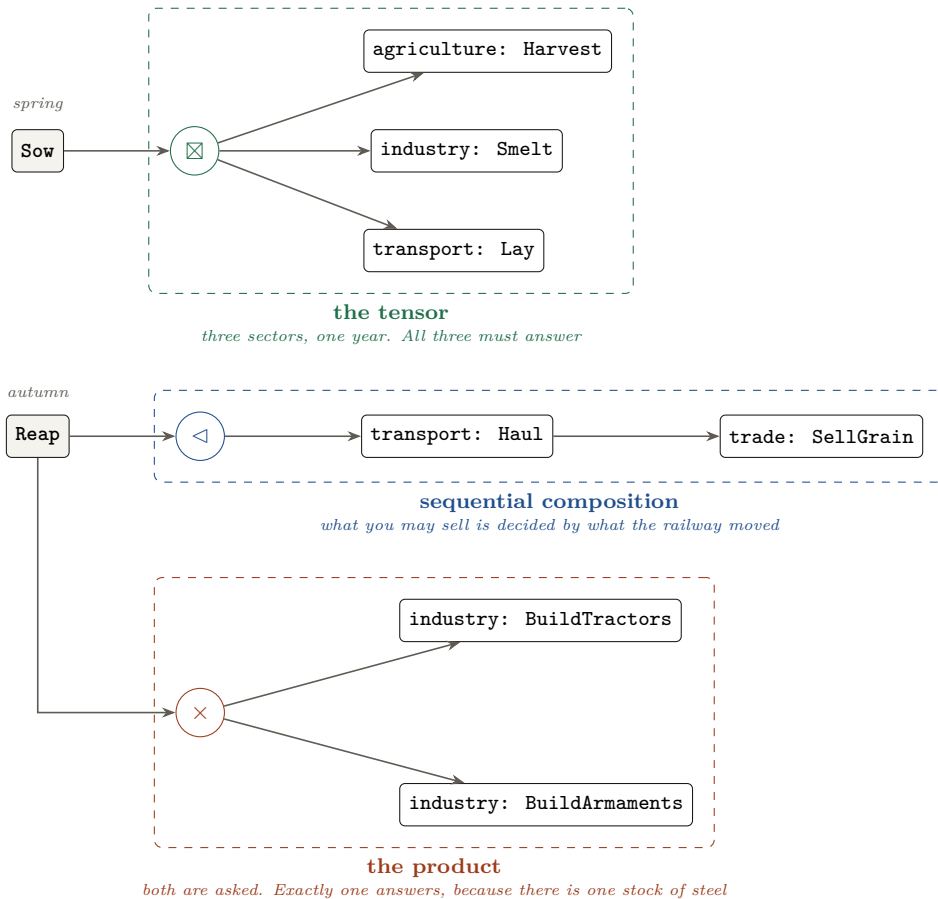
The purpose of this game is to illustrate Ghani’s theory of container combinators for event-driven programming. The game is the example. The algebra is the subject. As yet there is no “typed agent” in the game, but all the machinery has been built to easily add one. This is the obvious next step.

Recall the four combinator operators, defined by Ghani in *Containers for Typed Agentic AI*. The tensor $\boxtimes$ prompts both sides and requires a response from each. The sequential $\triangleleft$ prompts one side, and computes the second prompt from the first response. The product $\times$ offers both sides and takes a response from exactly one.

The sum $+$ takes a prompt belonging to one side or the other, and answers with that side’s own response. It is routing. We only use the first three in this game. Adding routing to the game is another future project.

This is a game about Stalin and his First Five-Year Plan. You are given the Soviet Union in 1928 and five years to industrialise it. You must also win the war in 1941.

This was a tragic episode of history and I don’t wish to make light of the story. It is for illustrative purposes only. It is also a history lesson.



2 The prompt

I am an agentic engineer. I generate code using agentic coding agents such as Claude Code, Grok and Codex. The following description is a rough approximation of the prompt I gave Claude to build the game.

You want to build tractors. The peasants are starving. More tractors means more grain, and peasants can be moved off the land to work on railways and in steel mills.

You need steel to build tractors. You also need steel to build armaments. Steel needs a mill. A mill needs a German engineer. An engineer has to be paid in hard currency. Hard currency comes from selling grain to the USA.

You need to build railways. You cannot sell the grain without railways. Railways need steel too, but they are not capital. They are *throughput*. Grain the railway cannot move rots at the sidings, whatever the harvest was. So track comes before tonnage. This is the sequential operator $\triangleleft$. What may be sold is computed from what the railway managed to move.

You can buy tractors and steel. But this will not mechanise the country.

You must prepare for war. In 1941 the country is invaded. Peasants serve as the soldiers, so the same population is both the workforce and the army. Steel not spent on tractors or train tracks can be spent on armaments instead. Fight with too little steel and you win by sending more men. Fight with more steel and send fewer peasants to die. Fail to arm at all and no amount of mechanisation saves you.

You must also feed the peasants. The labour force is one pool of people. They can work the fields, drive the tractors, work the mill or lay the track. But everyone eats, and only the first two grow anything. That is why moving a peasant into the mill costs you twice.

The Soviet Union is a centrally planned economy. How hard the villages are squeezed is a decision you make every spring. What is not taken is what they eat. Starvation in this game is not a scoring gimmick. It is the direct arithmetic consequence of the extraction and investment decisions made each spring and autumn, which is what it was for Stalin.

There are three goals. Mechanise the country, win the war, and starve as few people as possible.

3 Strategy

The game is more complex than it sounds, which is why Ghani's theory of container combinators comes in handy.

The game has been built, by Claude Code, and is playable. Just pull the repo and type `./play`.

`github.com/RaggedR/stalin`

There are various strategies. They disagree with each other. What follows is what happens when you try them.

A year is two turns, spring and autumn. In *spring* you fix where the hands go and how hard the villages are squeezed, *before* you know the weather. In *autumn* the harvest comes in graded, and you dispose of it knowing exactly what it was. A quota set in March against a harvest that fails in August is not a game mechanic. It is how a famine is administered. Five years is ten decisions, and then a reckoning.

3.1 Why the two costs are not independent

Combat power in 1941 is

$$m \cdot \text{manPower} + \text{steelPower} \cdot \sqrt{W \cdot m}$$

for m men and W tonnes of war reserve. A man is worth something holding nothing. A shell is worth nothing with nobody to fire it. And the second term grows as a square root, so the more steel each man already carries, the less the next tonne adds.

Men and steel are therefore complements, and partly substitutes. The same victory can be bought with more of one or more of the other. The price of the second is paid in lives.

The trap is that extraction feeds both sides of that expression with opposite signs. A peasant starved in 1932 is a soldier missing in 1941. So squeezing harder to build steel shrinks the army the steel was meant to protect. Push far enough and you lose the war holding the largest stockpile in Europe.

3.2 Build them or buy them

There are two roads to a tractor, and choosing between them is most of the early game.

Build them. Hard currency buys a German engineer. The engineer raises the mill. The mill makes steel. Twenty spare tonnes of steel opens the works. The works makes twenty tractors a year forever. Four steps, and the first tractor is years away.

Buy them. Ten in hard currency each, ready to drive. They need no engineer, no mill and no works. They build none of those either. Every one is bought again next time.

3.3 Seven plans

A plan keeps its letter for the whole game. So a sequence of ten letters is a strategy, and the space can be searched directly. All seven below were run at seed 1928. The numbers are what came out of the terminal.

strategy	letters	tractors	armaments	starved	fallen	dead	outcome
gentle armament	AJBEBJAJAJ	2	89	5	4	9	won
brutalist mech.	CMKEBIBJNJ	22	80	14	5	19	won, mechanised
no railway	KABEBIBJAJ	22	82	25	4	29	won, mechanised
partial armament	CABEBINJAA	42	34	21	14	35	won, mechanised
totally brutal	KJBEBJBJAJ	2	78	44	5	49	won
bought tractors	AMAMAFAMAF	18	0	3	57	60	<i>lost</i>
never armed	CAKEBINIAI	62	0	15	52	67	<i>lost</i>

Never arming loses. A beautifully mechanised country falls in 1941. Sixty-two tractors and the largest harvest on the board. But no more Russia.

Buying the tractors abroad is the humane road, and it is fatal. Three starve, the fewest of any line. But imported tractors build no mill, no engineers and no steel. So there is nothing to arm with, and Russia is destroyed.

Partial armament wins and bleeds. Thirty-four tonnes is enough to win. It is not enough to win cheaply. Fourteen fallen, against the best line's four. The war curve is smooth, so half an army is a victory paid for in men.

The railway should come before the steel. Grain the railway cannot move rots at the sidings. Peasants starve. The total number of dead is still less for no railway than for partial armament, because steel which would have been invested in the railway is invested in armaments.

Total brutality. Starving the peasants to build the army wins the war. But it also fails to mechanise the country.

3.4 A surprise

The parameters of the game were tuned, by Claude Code, so that the *brutalist mechanisation* strategy would be optimal. Starve the villages early. Spend it on rail, engineers and tractors. Then turn the surplus into an army. This was Stalin's strategy. It is morally uncomfortable and it was supposed to win.

It does not. At least not in terms of total numbers dead. The plan AJBEBJAJAJ wins for nine dead by doing almost nothing. It was found by machine search over the menu rather than by hand. Put hands in the mill. Buy the Germans once. Then smelt straight into armaments for four years. No railway. No tractors on purpose. And no famine beyond the baseline five who die whatever you do.

But with this strategy the country is still not mechanised at the end of the war. Only two tractors.

4 The Unix operating system, pipes, and CLI tools

In case you are a little rusty we review some basic ideas about the Unix operating system. This might be useful to Neil.

4.1 A process

A *process* is one running program, together with everything the kernel keeps on its behalf. Its memory. Its position in the code. Its identity – meaning which user it runs as. And a number.

A program is a file sitting on a disk, and does nothing. A process is that file in motion. Run the same program twice and you have two processes. They share nothing and cannot see each other.

The number is the process’s *pid*. It is how anything outside the process refers to it, and there is no other handle. To stop a program you send a signal to its pid.

Every process has a parent. The kernel is not a process. At boot it creates the first process, and that first process launches all the others.

On my MacBook Air the first process is called `launchd`. On Linux it is `init` or `systemd`. Its own parent is given as 0, which is not a real process.

4.2 User space and kernel space

A running machine is divided in two. Your code executes in *user space*. There it may do arithmetic, compare values, jump from one place in its own code to another, and read and write the memory that belongs to it. That is the whole of what it may do. It cannot address a disk, a network card, or another process’s memory. The physical processor is in a mode that forbids it. The *kernel* executes in kernel space, where every device and every process’s memory is addressable. The division is enforced by the hardware, not by convention.

There is exactly one interface between the two. It is the *system call*. It suspends your process, switches the physical processor into kernel mode, runs kernel code on your process’s behalf, and returns. Every effect a program has on the world outside its own memory is a system call. There is no other mechanism.

4.3 read and write

That interface could have been wide. It is in fact very narrow. The two calls that matter most are these:

```
read (n, buffer, count)  -- give me up to count bytes from the thing numbered n
write(n, buffer, count)  -- take these count bytes and put them in the thing
                          numbered n
```

Each takes a number identifying something already open, a place to put bytes or take them from, and how many. Between them they are very nearly the whole interface to the outside world. What that number is, and where it comes from, is the subject of two subsections below.

4.4 Everything is a file

Why should two calls be enough? Because of a design decision usually stated as a slogan. “Everything is a file.” A file on a disk, a terminal, a network connection, a source of random

bytes. All of them are presented to user space through the same interface. So the same two calls reach all of them.

The slogan overstates itself in one respect, and it is worth being precise about which. Things do differ in how they are *made*. A file is opened by name. A network connection has to be built by a short sequence of calls of its own. What they do not differ in is how they are *used*. Once the thing exists and you are holding its number, `read` and `write` are the whole vocabulary. The uniformity is at the point of use. That is the point that matters, because using is where one program meets another.

4.5 File descriptors

That number is a *file descriptor*. It is a small non-negative integer. It is an index into the *file descriptor table*, which the kernel keeps for each process, and whose entries point at the things that process has open. Nothing in the number says what kind of thing it is. The descriptor is an opaque handle. That is exactly why one program can be pointed at a file one day and at a network connection the next without being rewritten.

Three entries are filled in before a process begins, and it did not fill them in itself. Whoever started it did.

0	standard input	the stream the program reads
1	standard output	where its results go
2	standard error	where its complaints go

Start a program from a terminal and all three point at that terminal. That is why results and complaints appear jumbled together in one scrolling window. It is a property of the terminal, not of the program.

4.6 cat

Everything so far is enough to write a real Unix program. The program `cat` is named for *concatenate*. It copies its input to its output, unchanged, and does nothing else. That is its entire specification. It is almost exactly `read` and `write` in a loop.

```
char buf[4096];
for (;;) {
    n = read(0, buf, 4096);    -- take up to 4096 bytes from standard input
    if (n == 0) break;        -- 0 bytes means end of input; stop
    if (n < 0) exit(1);        -- negative means the read itself failed
    write(1, buf, n);          -- put exactly those n bytes on standard output
}
```

The real `cat` is longer. The extra length is all flags and error messages. That loop is the program. Look at what it does *not* contain. It never asks what 0 is attached to, or what 1 is attached to, because `read` and `write` provide no way to ask. A disk file, a keyboard, another program's output. The loop is identical and the program cannot tell the difference.

4.7 fork and exec

Two more system calls, and they are the two that make new processes.

The call `fork` *duplicates* the calling process. Where there was one process there are now two. They are identical in their memory, their open descriptors and their position in the code. They

differ only in the pid `fork` handed back. That is how each tells whether it is the original or the copy.

The call `exec` *replaces* the program running inside a process. Same process, same pid, same file descriptor table. But the old code and memory are discarded and a new program is loaded in their place.

Neither is a “run this program” call, and that is the point. To start a program, a process `forks`, and then the copy `execs`. The file descriptor table survives both, so the copy inherits the original’s descriptors. And in the gap between the two calls the copy is still running the original’s code, so it can rearrange its own table before replacing itself. Three subsections below turn on that gap.

4.8 The shell

The shell is not part of the kernel and has no privilege you do not have. It is an ordinary process running in user space with your identity, and its purpose is to start other processes. That is all it is. A process whose job is starting other processes.

4.8.1 The environment

A process carries one more thing. Its *environment* is a set of named strings held alongside its memory and its file descriptor table. They are called variables, but they are only ever text. Like the file descriptor table, the environment survives `fork` and `exec`. So a new process starts with a copy of the one that started it.

```
$ export GREETING=hello      -- add it to this shell environment
$ sh -c 'echo $GREETING'    -- a different process, started by this one
hello
```

That is the whole of it: named text, inherited downwards. The most used of them is `PATH`, which holds a list of directories; the shell searches them whenever it is looking for a program to run.

4.8.2 Globbing

A word containing `*`, `?` or `[...]` is a *glob*: a pattern to be matched against the names of files that exist. The symbol `*` stands for any run of characters, `?` for a single one, `[abc]` for one character out of a set.

The matching is done by the shell, before any program is started:

```
$ echo *.txt
log.txt notes.txt other.txt
```

The command `echo` prints its arguments and nothing else. So what it printed is exactly what it was given, namely three filenames. It never saw an asterisk. That is why every program accepts patterns, and why no program contains any code for matching them.

4.8.3 What the shell does with a line

Given a line, the shell does five things, and every one of them has already been described.

1. **Split the line into words.** The first word names what to run. The rest become the argument vector that program will receive.
2. **Find the program.** The first word is looked for in each directory named by `PATH`, in order, and the first match wins.
3. **fork.** The shell duplicates itself.
4. **exec.** The copy replaces itself with the program. The shell that read the line is still there, waiting for it to finish.
5. **Keep the exit status.** When the program finishes the kernel gives the shell a small number it left behind, by convention zero for success.

One command cannot work that way, and the reason is worth seeing. The command `cd` has to be built into the shell rather than being a program. A program runs in the copy, and the copy changing its own working directory would leave the shell exactly where it was. Anything that must alter the shell's own state has to be part of the shell.

A remark, because this is the mechanism behind something that catches people out with coding agents. Claude Code runs each shell command exactly as the shell runs a program. It forks, and the copy execs. A `cd` in that command changes the copy's working directory, and the copy then exits. So no command the agent runs can change the agent's working directory. The directory it was launched in is the directory it has for the whole session. That is why you must start it in the project you actually mean to work on.

4.9 Redirection

Now the gap between `fork` and `exec`. Writing `> log` after a command tells the shell to do one thing in the copy, before exec'ing. Open the file `log` and put it in slot 1.

```
$ echo hello > log      -- slot 1 is now the file, not the terminal
$ cat log
hello
```

Writing `2> errs` redirects descriptor 2 in the same way. Nothing was added to `echo` to make this work. The command `echo` cannot discover that it happened. It wrote to descriptor 1 exactly as it always does. Redirection is the shell's syntax and the shell's doing, from beginning to end.

4.10 Pipes

A *pipe* is a buffer the kernel will hand out on request. It comes with two descriptors. One to write into, one to read out of. The symbol `|` is the same redirection as `>`, with a pipe where the file was. The shell asks for a pipe, forks twice, puts the writing end in the left program's slot 1 and the reading end in the right program's slot 0, and execs both.

```
$ cat notes.txt | wc -l
3
```

The command `wc -l` counts the lines on its standard input. It was given no filename and does not know one exists.

Both programs run at the same time. The program `wc` does not wait for `cat` to finish. The buffer is finite, at 64×1024 bytes on Linux, and the kernel makes each side wait for the other

rather than growing it. When it is full, `cat`'s next `write` does not return until `wc` has consumed something. When it is empty, `wc`'s next `read` does not return until `cat` has produced something. Neither program contains a line of code about this.

Two things follow, and both are worth having in mind.

A pipeline over a very large file needs no more memory than a pipeline over a small one. At no moment does either process hold more than the buffer.

And the second program starts answering immediately rather than at the end. Take `cat huge.log | grep x`. The command `grep` is handed the first 64×1024 bytes as soon as `cat` has written them, and reports its first match from those, while `cat` is still reading. It does not need to wait for `cat` to finish. Nothing in either program arranges for it not to. That is simply what two processes and one finite buffer do.

4.11 Threads

The pipe in the previous subsection contained a piece of concurrency and did not draw attention to it. The programs `cat` and `wc` run at the same time. It is worth asking what that sentence can possibly mean. For this we need to understand threads.

The physical processor is handed out in slices.

Assume the machine has one physical processor. It executes one instruction stream. It has no notion of a process. To run many processes on it the kernel keeps a set of flows of control it considers *runnable*, and gives the physical processor to one of them. A hardware timer interrupts a few milliseconds later. The kernel saves that flow's registers and its position in the code, loads another's, and returns. The interrupted process is not asked and cannot tell.

That is *preemption*. The consequence to carry forward is a single sentence. A program's text does not mark the points at which it may be interrupted, because every point between two machine instructions is one.

Blocking is the other way to lose the physical processor.

When `wc`'s `read` finds the buffer empty, the kernel marks it *not runnable* and gives the physical processor to somebody else.

A blocked process consumes nothing at all. It is not a candidate for scheduling until the thing it waits on becomes ready. Then the kernel makes it runnable again and it resumes inside `read`, as though that call had merely been slow.

This is the default behaviour of nearly every system call that touches the world. It is why the programs in the previous subsections could be written as straight lines, with no mention of waiting anywhere in them.

What the kernel keeps on a process's behalf.

The bookkeeping falls into two lists, and the difference between them is what this section is about.

The first list is what the process *owns*. Its address space, meaning the tables that map the addresses appearing in its code to real memory. Its file descriptor table. Its identity – meaning which user it runs as. Its pid.

The second list is what the kernel needs in order to *run* it. Its position in the code, its registers, and its stack.

Only the second list is required to give a process a slice of physical processor.

The first is what makes a process expensive. Creating one means building page tables. Switching from one to another means the physical processor's address-translation caches now describe the wrong process and must be refilled. The first list is also what isolates. Two processes cannot see each other because neither address space contains a mapping to the other.

A thread is the second list on its own.

A *thread* is a flow of control. A position in the code, a set of registers, and a stack. Nothing else. Every process begins with exactly one thread, and may ask the kernel for more. Threads belonging to the same process share every item on the first list. They share one address space, so every variable one of them can reach, all of them can reach. They share one file descriptor table, so anything one of them opens is open for all of them. They share one identity and one pid. What they do not share is the second list. Each has its own position in the code, its own registers, and its own stack.

The kernel schedules threads, not processes. Blocking is therefore per-thread. One thread stuck inside `read` leaves its siblings runnable, and that is the whole reason to want more than one.

Threads are much cheaper than processes. All they need is their own stack, and a stack is typically only a few megabytes of reserved address space.

4.12 Exit status, and the shell's control flow

The small number a program leaves behind is what the shell builds its control flow on.

- `;` run the next command regardless of the status.
- `&&` run the next command only if the previous one exited zero.
- `||` run the next command only if the previous one exited non-zero.
- `&` start this command and do not wait for it, so several can run at once.

We could say a good deal more about all this. It is not really relevant to the game `stalin`.

4.13 Shell scripts

A file of such lines is a shell script, and running it is the same as having typed them. So a pipeline that worked once can be kept and named. And a script is started like any other program. It is found on `PATH`, `forked`, `execed`, and its status collected. So a script can appear in a pipeline itself, on either side, and nothing that uses it needs to know it is a script.

4.14 Three channels, not one

Gather up what a program actually offers the world outside it. There are three separate channels, and they carry three different kinds of thing.

Channel	Carries	Read by
descriptor 1 (stdout)	the results	the next program in the pipeline
descriptor 2 (stderr)	the narration	a person
the exit status	the verdict	the shell

Keeping them apart is what makes a program usable by another program. A pipe takes descriptor 1 and nothing else. So whatever reads from a program receives its results, and never has to sort them out from error text. Putting results and narration both on descriptor 1 is the commonest way to make a program unusable in a pipeline.

4.15 Manual pages, and what is not checked

None of the above is checked by anything. The interface between two programs is bytes. Any two can be joined. That is the strength. Nothing verifies that the composition means anything. That is the cost. What a program promises is written in its manual page and nowhere else. So the only thing enforcing it is that you read it.

Here is what that costs. You want to build only if the log has no errors in it. So you write this.

```
$ grep -c ERROR log && ./build
```

The operator `&&` runs the next command only if the previous one exited zero. And `grep` exits zero when it *finds* something. So the line does the exact opposite of what you wanted.

```
$ grep -c ERROR dirty.log && ./build
1
  building                -- errors present, grep exits 0, so it builds
$ grep -c ERROR clean.log && ./build
0
  -- no errors, grep exits 1, so it does not
```

It builds exactly when the log is dirty and refuses exactly when the log is clean. Both halves of `grep`'s behaviour are documented. Neither is enforced. The shell did precisely what it was told. Nothing anywhere reports a problem.

What you wanted was `grep` answering *did you find anything* rather than *how many*. That is what `-q` is for. It prints nothing at all and reports only in its exit status. Zero if it found at least one match, one if it found none. Put the test where the shell can actually see it, and use `||` so the build runs when nothing was found.

```
$ grep -q ERROR log || ./build
```

Notice that `-c` was carrying that same exit status all along. The count it printed was never the thing `&&` was reading.

A note before leaving this. Claude is an absolute master at writing shell scripts, and this is genuinely a place where you do not need the details. Ask Claude for a script that builds only when the log is clean and you will get a correct one. What you do need is the concept.

None of this is enforced, and this is a deliberate design decision. The only thing holding any of it together is “developer discipline”. Somebody read the manual page. At this level of the stack that is the whole of the enforcement there is.

4.16 The dictum

Let's summarise what we have. One uniform interface reached by `read` and `write`. Three separate channels. A pipe that joins one program to another. With that on the table, the philosophy reads as a consequence of the machinery rather than as an aspiration. Doug McIlroy wrote the pipe into Unix v3 in 1973. He put it in three sentences.

Write programs that do one thing and do it well. Write programs to work together.
Write programs to handle text streams, because that is a universal interface.

Mike Gancarz later expanded this into nine tenets. Small is beautiful. Each program does one thing well. Build a prototype as soon as you can. Choose portability over efficiency. Store

data in flat text. Use software leverage. Use scripts to increase leverage and portability. Avoid captive user interfaces. Make every program a filter.

The last is the one that matters here, and it is the least obeyed. A *filter* is a program that reads a stream and writes a stream. Its input is not a dialogue. Its output is not a report to a human. The programs `cat`, `wc` and `grep` are filters. That is why the examples above could be assembled without any of them having been designed for each other.

4.17 A CLI tool is already a dependent function

One last observation, and it is the one this note turns on.

A command-line tool receives one prompt, its argument vector. It produces a response whose *type depends on which prompt it was*. The command `git log` answers with a list of commits. The command `git status` answers with a working tree. Those are not the same kind of thing, and which kind you get is fixed by the word after `git`. That is $(p : P) \rightarrow Rp$, with P the subcommands and R the result type of each. The interface (P, R) is what `--help` prints.

Notice what is *absent*. No event loop, no callbacks, no waiting, no concurrency. A CLI starts, is prompted once, answers, and exits. It is nonetheless event-driven in the sense that matters. What makes a program event-driven is the prompt-indexed response type, not the waiting. Strip the loop away and this is what remains.

Now the qualification, and it is sharp enough to be worth the paragraph. That reading is *correct*, and it is entirely ours. Ask what the machine sees and the answer is that `git` received an array of strings and produced a sequence of bytes on descriptor 1, and that this is true of every subcommand equally. There is no level at which the response to `git log` and the response to `git status` have different types, because at the only level Unix knows about they have the *same* type, which is bytes.

Put it in the notation of containers. The container Unix actually implements is

$$P = \text{String}^* \qquad Rp = \text{Bytes} \quad \text{for every } p.$$

A container whose R ignores its argument is not a dependent thing at all. It is an ordinary function space. The dependency in “`git log` answers with a list of commits” is a fact about *what the program does*. It is nowhere in the interface. It lives in a manual page, in the habits of whoever wrote the program, and in the head of whoever is reading the output.

So hold on to the pair, because the rest of this note is the same pair four more times. And hold on to the deficit as well, because that is what the four remaining layers are for. Each one takes an R that does not depend on p and makes it depend on p a little more, over a little less. §18 keeps the score.

5 The Claude Code harness

Claude Code is a CLI tool that executes other CLI tools. When it runs a command it does what the shell does. It **forks**. The copy **execs**. The copy inherits its parent's identity.

That's it. It runs with the permissions of the user that executed it. It can read and write any file that user can read and write, and it can execute any command that user can execute.

As we already mentioned, Claude Code is itself just another CLI tool. This is worth stopping and reflecting upon. It is what makes this design so beautiful.

You run it with a prompt as its argument, and it answers on standard output.

```
$ claude -p 'Reply with exactly the four words: this is a reply'
this is a reply
```

Claude Code also calls other CLI tools. That is nearly the whole of what it does. It reads a prompt, decides which commands to run, forks and execs them, reads what they wrote, and answers.

You can set him up with limited privileges, a handful of CLI tools that he has permission to operate and nothing else.

Now think of him as a container. The prompts you may send are the things you may ask for. The response you get back has a type that depends on which prompt you sent. And the interface is the list of tools he is permitted to use.

6 HTTP and JSON

We don't assume that you know anything about web app development, so we review here the HTTP protocol and the JSON datatype.

The simplest kind of web app runs a database on a server. A client, somewhere else entirely, reads from that database and writes to it. Every request is one or the other. That is the whole arrangement, and it is all this section needs.

The examples below all run against a toy server whose database is a list of books. It has nothing to do with the game. It exists to introduce TypeScript syntax.

6.1 A request and a response

HTTP is a request/response protocol. A client sends a request. A server sends a response. Nothing else happens. Here is one complete exchange, the request and the response.

```
> GET /books/1 HTTP/1.1
Host: 127.0.0.1:8099
Accept: */*

< HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 53

{"id": 1, "title": "Diaspora", "author": "Greg Egan"}
```

That's it. A request is a *method* (GET), a *path* (/books/1), some *headers* (blue), and optionally a *body* (none in this example). A response is a three-digit *status* (200), some headers (blue), and a body (green).

6.2 Deliberately simple

The designer of the HTTP protocol, Roy Fielding, made the same kind of design decision that Unix made. And it paid off. Just as every server in the world today runs Unix, every web app in the world today communicates via HTTP.

6.3 The method says what kind of thing you are doing

There are four types of HTTP request. We only use two of them.

Method	Means
GET	read something, change nothing
POST	create something

A simple web app runs a database on the server. When the server receives a `GET` request it reads the database and returns the result to the client. When the server receives a `POST` request it writes to the database.

In our “toy example” the database simply contains a collection of books with their title and author. Each book has a unique ID number.

Here is an example of reading the book with ID 1 from the database.

```
$ curl -X GET http://127.0.0.1:8099/books/1
{"id": 1, "title": "Diaspora", "author": "Greg Egan"}    -- 200 OK
```

It is worth breaking down the address of a single book.

`http://127.0.0.1:8099/books/1`

Part	Meaning
<code>http</code>	the protocol to speak
<code>127.0.0.1</code>	the machine, which here is this one, known as <i>localhost</i>
<code>8099</code>	the port, which says which program on that machine
<code>/books/1</code>	the path, which says which thing that program should answer about

The command `curl` is just another Unix CLI tool. It sends one HTTP request across the network and prints the response on standard output. Note that since “everything is a file” the Unix kernel handles all the networking, and `curl` just thinks it is writing to a file.

Here is an example of adding a new book to the database.

```
$ curl -X POST http://127.0.0.1:8099/books \
-H 'Content-Type: application/json' \
-d '{"title": "Permutation City", "author": "Greg Egan"}'
{"id": 2, "title": "Permutation City", "author": "Greg Egan"}    -- 201 Created
```

6.4 JSON

JSON is short for JavaScript Object Notation. It is a recursive text format for data. Just like HTTP it is deliberately simple. It has four types of “atomic” data: numbers, strings, booleans and `null`. Then it has two recursive constructors: array and object.

```
value ::= string | number | boolean | null
       | '[' value* ']'
       | '{' (string ':' value)* '}'
```

6.5 A request and a response, as values

Everything above is text on a wire. Inside a TypeScript program the two halves of the exchange are ordinary values, and they have names. A **Request** is one arriving HTTP request. A **Response** is one outgoing HTTP response. Both are supplied by the runtime, so no library has to be installed to use them.

A **Response** is built from the three things the exchange above already showed. A body, a status number, and a set of headers.

```
const body = JSON.stringify({ id: 1 });      // the body, as a string

const res = new Response(body, {
  status: 200,
  headers: { "content-type": "application/json" },
});

console.log("body   :", body);
console.log("status :", res.status);
console.log("header :", res.headers.get("content-type"));
```

```
$ deno run r.ts
body   : {"id":1}
status : 200
header : application/json
```

The body is a string. To keep things simple, we will assume that the bodies of our requests and responses are well formed JSON. The status is the number whose meaning was given above. So constructing a **Response** is nothing more than writing down those three fields, and a server handler is a function that returns one.

The body of a **Request** arrives on a socket. Getting it means a **read**, and §4.11 said what a **read** does when nothing is there yet. It blocks. That is true however small the body is. A server may not block, so it cannot simply wait. Section 11 is about what it does instead.

7 TypeScript syntax

It is an unfortunate accident of history that JavaScript became the “default” language for web app development. TypeScript is an unfortunate attempt to “bolt on” a type system to JavaScript after the event.

This is a “crash course” in TypeScript, and I am also learning as I write here. It is a bit of a head twister, and it bears almost no resemblance to Haskell or Lean.

7.1 Basic types

- The atomic types are `number`, `string`, `boolean`, `null` and `undefined`.
- A string in quotes may be used as a type. The type `"failure"` has exactly one inhabitant, namely that one string.
- The type `never` has no inhabitants at all.
- The type `A | B` is a value that is an `A` or a `B`. It is an untagged union.
- The type `{ id: number; title: string; author: string }` is a *record*. A value of it is a dictionary in the Python sense, with those three keys. In our running example this is the type of a book, and the note gives it the name `Book`.
- The list type can be written using two different notations. `Array<Book>` and `Book[]` both mean the same thing, a list whose elements are all books. It is easier to see from the first notation that this is one type taking another type as an argument.
- The signature `<T, R>(x: T) => R` is a generic function.
- The keyword `export` in front of a definition means that other files may use it. Without it, the definition is private to its own file. This is an unhappy collision with the shell’s `export` of §4, which adds a variable to the environment. The two have nothing to do with each other.

7.2 The weird stuff

- The type `unknown` is a value about which nothing is known. Nothing may be done to it until you have established what it is.
- The type `Record<string, unknown>` is a dictionary again, but this time with arbitrary keys and nothing at all known about the values.
- The keyword `any` switches the type checker off for that value entirely.
- The notation `u as T` says “trust me, this is a `T`”. This is very upsetting for somebody in formal verification. It is called a *type assertion*.

7.3 Type guards

The notation `u as T` above asserts a type and checks nothing. There is a second notation that checks, and it is the one the wire code in this note is built on.

```
export const isRecord = (u: unknown): u is Record<string, unknown> =>
  typeof u === "object" && u !== null && !Array.isArray(u);
```

The word `is` is a keyword. It may appear only in the return type of a function, written `u is T` where `u` is one of that function's own parameters. It says that the function returns a boolean, and that a true answer means `u` is a `T`. A function declared that way is called a *type guard*.

At run time this changes nothing. At compile time the checker narrows `u` to `T` wherever the answer was true. Every validator in §10 and in the game's wire code is a function declared this way.

7.4 Subtypes

- In TypeScript we can talk about one type being a *subtype* of another. For example the type `"failure"` is a subtype of `string`. Every type is a subtype of `unknown`, and `never` is a subtype of every other type.
- The type `Extract<Q, S>` keeps only those members of the union `Q` that are subtypes of `S`, and throws the rest away.
- The notation `Book[K]` tells you the type of the value stored at field `K` of the record type `Book`. It is called an *indexed access type*. So `Book["id"]` is `number`, and `Book["title"]` is `string`. Note that the square brackets here mean something quite different from the ones in `Book[]`, which is why this note writes `Array<Book>` instead.
- The constraint `K extends U` says that `K` must be a subtype of `U`.

7.5 The powerful stuff

- The notation `X extends Y ? A : B` is a *conditional type*. It computes a type by testing `X` against `Y`. This is the construct everything else in this note rests on, and it gets a subsection of its own below.
- The notation `{ [K in U]: F<K> }` is a *mapped type*, with one field for every member of the union `U`. This one also gets a subsection of its own below.

7.6 async functions

The type `Promise<T>` is a computation of a `T` that has not finished yet. The keywords `async` and `await` are how one is made and how one is consumed.

7.7 On the level of values

- The keywords `interface` and `type` both name a type, the first restricted to records.
- The declaration `const x: T = e` introduces a constant called `x`, gives it the type `T`, and sets it equal to the value `e`. The type may be left out. Writing `const n = 3` gives `n` the type `number`, because `3` is a number and the checker can see that.
- The expression `(x) => e` is a lambda expression.
- The expression `x ?? y` is equal to `x`, unless `x` is `null` or `undefined`, in which case it is equal to `y`.
- The expression `c ? a : b` is the ordinary conditional expression on the level of values.

8 Structural typing

A type describes what fields a value has. Any value with those fields has that type, whether or not anybody said so. Nothing has to be declared and nothing has to be constructed.

For unions that has a cost, and an example makes it plain. Suppose a length is either in metres or in feet.

```
type Metres = { value: number };
type Feet   = { value: number };
type Length = Metres | Feet;
```

Those two are the same type. A value with one numeric field called `value` is a `Metres`, and it is equally a `Feet`, and the checker will let you pass either one where the other is expected. Given a `Length` there is no test that tells you which you have, because there is nothing to test.

In Haskell you would write `data Length = Metres Double | Feet Double`. The constructor is itself the answer. TypeScript has no constructors, so if you want to tell the two apart you must put something there to look at.

```
type Metres = { unit: "m"; value: number };
type Feet   = { unit: "ft"; value: number };
```

Now `x.unit === "m"` answers the question. But notice what `unit` is. It is an ordinary field holding an ordinary string. The language did not provide it. We added it, and we have to keep adding it, and everything that wants to tell metres from feet has to read it.

8.1 A tagged union writes down a finite choice

Once every case carries a tag, the checker can use it. Stay with lengths, and give the two cases different data so that there is something to get wrong.

```
type Length =
  | { unit: "m"; metres: number }
  | { unit: "ft"; feet: number };
```

9 Conditional types

Everything this note does depends on one construct, so it is worth going slowly.

A *conditional type* is a function from types to types. It is a type constructor. It is written with the same punctuation as an ordinary conditional expression, but it lives one level up, and the checker evaluates it before the program runs.

Stay with the shelf of books. A book has three fields, and their names are `id`, `title` and `author`. Those three names form a finite union.

```
type Field = "id" | "title" | "author";
```

9.1 A function from types to types

Now notice that the three fields do not hold the same kind of thing. An `id` is a number. A `title` and an `author` are strings. That fact can be written down as a function, on the level of types.

```
type FieldType<F extends Field> =  
  F extends "id" ? number : string;
```

The word `extends` appears twice there. Both times it means the same thing, namely *is a subtype of*. What differs is what is done with the answer.

In `<F extends Field>` it is a *requirement*. The type `F` must be a subtype of `Field`, which here means one of `id`, `title` or `author`. It ranges over the field names, which are the keys of the dictionary. It does not range over the types of the values stored under those keys.

In `F extends "id" ?` it is a *question*. Is `F` a subtype of `"id"`? That type has exactly one inhabitant, so the question amounts to asking whether `F` is `"id"` itself. The answer selects one of the two branches. If `F` is not `"id"` then it must be `"title"` or `"author"`. The value stored under either of those keys is a string, so the second branch gives `string`.

9.2 Question for Orestis

As a professional type theorist, what do you make of this nonsense? From a Haskell perspective it is baffling.

9.3 A function on the level of values

The result type of the function below is computed from its argument type. It looks a little bit like a dependent type, but that is not quite what it is.

```
function field<F extends Field>(b: Book, f: F): FieldType<F> {  
  return b[f] as FieldType<F>;  
}  
  
const n = field(book, "id");      // n : number  
const s = field(book, "title");   // s : string
```

That is the whole trick. The type checker read the string "id". It computed the type `FieldType<"id">`. It concluded that `n` is a number. Knowing that is what tells the checker which operations on `n` are legal.

9.4 The fake container

Compare this with the introduction. An interface was a pair there. The prompts you may send, and for each prompt the type of response it can receive.

Here `Field` is the type of the prompts, and `FieldType` is the type of the response for each prompt. So `(Field, FieldType)` is a container, and `field` is an inhabitant of it.

9.5 Another question for Orestis

To be honest, I don't understand this and would like you to explain it to me.

9.6 Two limits

- **The index must be finite.** `Field` has three members, so `FieldType` can be written out as a case split, here a single test and a default. A rule indexed by the natural numbers could not be written this way at all. Over a finite index a dependent type is just a table, and a table is something TypeScript can write down.
- **The argument must be a literal the checker can see.** In `field(book, "id")` the string "id" is written in the source. So the checker infers `F` to be the singleton type "id", and `FieldType<"id">` computes to `number`.

That second limit deserves its own example, because it is the one that bites in §19. Suppose the field name is not written in the source. Suppose it arrives from somewhere else, and all the checker knows is that it is one of the three.

```
declare const anyField: Field;      // one of the three, but which is not  
    known  
const m = field(book, anyField);  
const u: number = m;
```

In English, that block says three things. There is a field name, and it is one of the three (`id`, `title`, `author`), but nobody is saying which. Ask `field` for the value stored under it, and call the answer `m`. Then try to declare that `m` is a number.

The last line is the problem. It does not convert anything and it does not check anything at run time. It is a claim, made to the checker, that `m` has type `number`. If the checker agrees, the program compiles. If it does not, we learn what type the checker thinks `m` actually has.

```
$ deno check shelf.ts
TS2322 [ERROR]: Type 'string | number' is not assignable to type 'number'.
const u: number = m;
      ^
```

In this case the type checker thinks `m` is of type `string | number`. It cannot be declared a `number`, because as far as the checker knows it might be a `string`.

Why is `m` not a `number`? Because the checker was never told which field name it had.

Write `field(book, "id")` and the checker sees the word `"id"` sitting there in the source, so it takes `F` to be `"id"`. Write `field(book, anyField)` and all it knows is that the name is one of the three, so it takes `F` to be all three at once.

It then works out `FieldType` for each of them.

```
FieldType<"id">      is number
FieldType<"title">   is string
FieldType<"author">  is string
```

Put those three answers together and `m` could be a `number` or it could be a `string`. That is what `number | string` means, and it is why the last line was refused.

So the dependency only works when the checker can see which name it is, and the checker only ever sees the source. A name that is picked while the program is running cannot reach it.

9.7 Question for Orestis

How does Lean solve this problem?

9.8 Recovering the name

To get it back you write a `switch` with one branch per name. Inside `case "id":` the checker knows the name is `"id"` again, so `FieldType<"id">` is `number` once more. §19 pays exactly that price, and a type assertion in each branch on top of it.

9.9 Mapped types, which make a table total

The second construct to go slowly over is the one that builds a record with a field for every member of a union.

Keep the same three field names.

```
type Field = "id" | "title" | "author";
```

Suppose we want, for each field, a function that reads that field out of a book. There should be exactly three of them, and each should return the right kind of thing. A mapped type says so.

```
type Getters = { [F in Field]: (b: Book) => FieldType<F> };
```

Read `[F in Field]` as “for every `F` in `Field`”. It generates one field per member, and the field’s type may mention `F`. So `Getters` is a record with three fields, and their types are not the same as each other.

```
const getters: Getters = {  
  id: (b) => b.id,           // must return number  
  title: (b) => b.title,     // must return string  
  author: (b) => b.author,   // must return string  
};
```

That version compiles. Now the point of the whole thing. The record is *total by construction*. Try leaving one of the three out.

```
const getters: Getters = {  
  id: (b) => b.id,  
  title: (b) => b.title,  
}; // author is missing
```

```
$ deno check getters.ts  
TS2741 [ERROR]: Property 'author' is missing in type  
    '{ id: (b: Book) => number; title: (b: Book) => string; }'  
    but required in type 'Getters'.
```

The object with only two fields is not a `Getters`, because `Getters` has a field for every member of `Field` and `Field` has three members. So the program does not compile.

There is no default branch to fall through and no dispatch to forget. A whole class of mistake becomes unwritable rather than merely untested.

That is the second half of the trick. A conditional type says what the response to each prompt should be. A mapped type insists that every prompt has one. Together, over a finite union, they are a dependent function written as a table, and §19 builds the game’s five servers out of exactly this.

10 Compiling down to JavaScript

TypeScript compiles to JavaScript, and JavaScript has no types. So the types are checked, then deleted, and ordinary JavaScript runs. No value carries its type at run time, and no check can be derived from a type.

That one fact is the subject of this section. It is why a cast is not a check, and why a value from outside has no type until you give it one.

The system is deliberately unsound. This is the sentence to carry into the rest of the note. TypeScript does not claim that a well-typed program cannot go wrong. It claims to catch a useful class of mistakes.

10.1 Types are checked before the program runs

TypeScript is a type system over JavaScript. The types are annotations. You write them. A checker reads them. Then they are removed and ordinary JavaScript runs.

```
interface Book { id: number; title: string; author: string }
const b: Book = { id: "1", title: "Diaspora", author: "Greg Egan" };
```

```
$ deno check t1.ts
TS2322 [ERROR]: Type 'string' is not assignable to type 'number'.
const b: Book = { id: "1", title: "Diaspora", author: "Greg Egan" };
    ~~
```

Nothing ran. The mistake was found by reading the program. That is the whole point of having a checker. It is the pleasant half of the story.

10.2 A cast is a promise, not a check

The types are *erased*. The program that executes has no representation of them and no check derived from them. So the `as` operator is not a conversion. It is an assertion. The checker accepts it without verifying, and the runtime cannot verify it at all.

In the example below, `toFixed` is a method that rounds a number to a given number of decimal places. It exists on numbers and on nothing else, which is what makes it a useful test.

```
const raw = '{"id":"1","title":"Diaspora","author":"Greg Egan"}';
const b = JSON.parse(raw) as Book;    // a promise, not a check
console.log(b.id.toFixed(0));         // compiles: id is declared number
```

```
$ deno check t2.ts    -- passes. Nothing is wrong as far as the checker knows.
$ deno run    t2.ts
TypeError: b.id.toFixed is not a function
```

The checker was told `id` is a number and believed it. The `id` was a string all along. The error surfaced at the first place that actually used it. That may be a long way from the cast, and in a real program is likely to be somewhere else entirely.

10.3 unknown is what you want at a boundary

Two types describe a value you know nothing about, and they behave in opposite ways.

The type `any` switches the checker off. Every operation on an `any` is permitted.

```
const raw: any = JSON.parse('{ "id": "1", "title": "Diaspora", "author": "Greg Egan" }');
console.log(raw.id.toFixed(0));
```

```
$ deno check anyex.ts      -- passes, no complaint
$ deno run    anyex.ts
TypeError: raw.id.toFixed is not a function
```

The `id` was the string `"1"` all along. Nothing objected, because nothing was looking.

The type `unknown` does the opposite. It permits nothing until you have established what you have.

```
const raw: unknown = JSON.parse('{ "id": "1", "title": "Diaspora", "author": "Greg Egan" }');
console.log(raw.id);
```

```
$ deno check unknownex.ts
TS18046 [ERROR]: 'raw' is of type 'unknown'.
```

That refusal is the useful thing. The type `any` would have compiled and then failed later, in the manner of the previous subsection. The type `unknown` makes the boundary visible. It refuses to let you cross it accidentally.

10.4 Validate, do not cast

So how does a value we know nothing about honestly become one whose type we know? By code that looks at the fields. The signature to aim for is `unknown` in, and the type *or nothing* out.

```
function asBook(u: unknown): Book | null {
  if (typeof u !== "object" || u === null) return null;
  const o = u as Record<string, unknown>;
  if (typeof o.id !== "number") return null;
  if (typeof o.title !== "string") return null;
  if (typeof o.author !== "string") return null;
  return { id: o.id, title: o.title, author: o.author }; // rebuilt, not asserted
}
```

Try it on three documents.

```
asBook({ id: 1, title: "Diaspora", author: "Greg Egan" })  
// { id: 1, title: "Diaspora", author: "Greg Egan" }  
  
asBook({ id: "1", title: "Diaspora", author: "Greg Egan" })  
// null -- the id is a string  
  
asBook({ id: 1, author: "Greg Egan" })  
// null -- there is no title
```

11 `async`, `await`, and the event loop

This section contains one fact, and the rest of it is consequences. The keyword `await` is a continuation-passing transformation that the compiler performs and then hides. Three things the rest of this note needs follow from that single mechanism. The points at which one computation can be interrupted by another become countable. The four combinators of the algebra become expressible. And a two-line function body turns out to be a container morphism.

11.1 One flow of control, and it may not stop

A server holds many conversations at once, most of them idle most of the time, so a single flow of control that blocks can serve only one of them.

That is the whole difficulty. Section 4.11 said what blocking costs. A blocked process is not runnable, and the kernel will not schedule it again until the thing it waits on is ready. Inside a server there is one flow of control and it is inside your function, so one blocking call stops not that conversation but all of them.

The design used by `nginx`, by Node and by Deno keeps one flow of control and never lets it block. Every operation that would have waited is written in three parts instead.

1. **Start it.** Ask for the thing, and do not wait for it.
2. **Give the flow of control back.** Stop, so that the other conversations can be served.
3. **Arrange to be resumed.** Leave behind what should happen when the response arrives.

Something has to hold those arrangements. That something is the *event loop*. It is a set of outstanding resumptions, each of them waiting on one thing, together with a way of asking which of those things are ready now. The loop takes one that is ready, runs the piece of work it has earned, and asks again. It never blocks on any particular conversation because it never commits to one.

11.2 How it actually works

The runtime owns that loop and you never write it. You hand it one function, and it calls that function once for each request that arrives. Here is a whole server.

A reminder from §6.5. A **Request** is one arriving HTTP request and a **Response** is one outgoing HTTP response. A **Response** is built from a body, a status number and a set of headers. A handler is a function from the first to the second. And we are assuming throughout that the bodies of our requests and responses are well formed JSON.

```
const json = (body: unknown, status: number) =>
  new Response(JSON.stringify(body), {
    status,
    headers: { "content-type": "application/json" },
  });

const census = async () => ({ fields: 30, drivers: 22, dead: 11 });

Deno.serve({ port: 8099 }, async (req) => {
  const body = await req.json();
  if (body.tag !== "Census") return json({ error: "not a prompt I accept", got:
    body }, 400);
  return json(await census(), 200);
});
```

Three lines of that are the handler. The rest builds a response.

The keyword `await` appears three times and the keyword `async` twice. Section 11.5 is about what they do. Until then, read `await` as the point at which this function stops and is continued later.

Now send it a prompt it accepts, and then one it does not.

```
$ curl -si -X POST http://127.0.0.1:8099/ -d '{"tag":"Census"}'
HTTP/1.1 200 OK
content-type: application/json

{"fields":30,"drivers":22,"dead":11}

$ curl -si -X POST http://127.0.0.1:8099/ -d '{"tag":"Nonsense"}'
HTTP/1.1 400 Bad Request
content-type: application/json

{"error":"not a prompt I accept","got":{"tag":"Nonsense"}}
```

Two things in that handler matter for everything below. The response is computed from the prompt, so what comes back depends on what went in. And the function stops twice before it answers, once for the request body and once for the response, which is the reason a loop has to exist at all.

11.3 Inversion of control

This arrangement is called *inversion of control*. An ordinary program calls a library and the library returns an answer. Here the library calls the program.

The library in question is the server. The call `Deno.serve` is the whole of it. It owns the socket, it owns the loop, and it owns every connection that is open. The one thing it does not have is what to answer, so that is the piece you hand to it.

You pass your function to the library as an argument. So the server is the library plus your one function. Your function is never reached by any line you wrote, and the loop appears nowhere in your source.

11.4 A promise is the pending computation as a value

An operation that has been started and has not yet finished needs something to stand for it. The loop has to be able to say which of them are ready. Your code has to be able to say what should happen when one of them is. So the pending computation is made a value in its own right, and it has a name.

<code>Promise<T></code>	a computation of a T that has not finished yet
<i>pending</i>	no response yet
<i>fulfilled</i>	holds a T
<i>rejected</i>	holds an error

The type constructor `Promise` comes with the two operators one would expect. There is a unit, taking a value to a promise already holding it. There is a bind, taking a promise and a function from its value to a second promise, and giving a promise of the second. TypeScript writes the unit as `async` and the bind as `await`.

11.5 What `await` actually does, which is not waiting

The word is a lie, and the lie is the feature.

At an `await`, the function *suspends*. Everything after that point in the body is registered with the event loop as the continuation of that promise. Control returns to the loop, which is free to run every other conversation the server is holding. The frame is kept alive somewhere other than the stack. When the promise settles, the loop resumes that frame, at that line, with the value in hand, and execution proceeds down the rest of the body as though nothing had happened.

Here are two functions that each suspend once, started one after the other.

```
async function slow(name: string, ms: number) {
  console.log(`${name}: before`);
  await new Promise(r => setTimeout(r, ms));
  console.log(`${name}: after`);
}
slow("A", 20);
slow("B", 10);
console.log("both started, neither finished");
```

```
$ deno run d1.ts
A: before
B: before
both started, neither finished
B: after
A: after
```

Read the order. Neither call has reached its second `console.log` by the time the last line of the program runs. Both are suspended at their `await`, and the flow of control has gone back to the loop, which had nothing else to do but finish the program.

So the text of `slow` runs top to bottom and its execution does not. Between the two lines of one call, the whole of the other call and the rest of the program ran. There is no sleeping anywhere in the machine, and the flow of control was never idle.

Two properties follow, and the code in this note leans on both.

The value is typed. At the level of types `await` takes a `Promise<T>` to a `T`, so a function can say in its type what it produces.

```
async function titleOf(id: number): Promise<string> {
  const res = await fetch(`http://127.0.0.1:8099/books/${id}`);
  const book = await res.json();
  return (book as Book).title;
}
```

One distinction before moving on, because the two words are easy to run together. Although they are similar concepts, a suspended `async` function is not a blocked process as in §4.11. A blocked process is not runnable and the kernel will not schedule it. A suspended function is a data structure inside a process that is running perfectly well and answering other requests.

12 How Deno runs a CLI tool

The TypeScript web server Deno is itself a CLI tool, and it can execute other CLI tools.

```
const cmd = new Deno.Command("sh", {
  args: ["-c", "echo on stdout; echo on stderr >&2; exit 3"],
  stdout: "piped",
  stderr: "piped",
});

const { code, stdout, stderr } = await cmd.output();
const dec = new TextDecoder();

console.log("exit status :", code);
console.log("slot 1      :", dec.decode(stdout).trim());
console.log("slot 2      :", dec.decode(stderr).trim());
```

```
$ deno run --allow-run cmd.ts
exit status : 3
slot 1      : on stdout
slot 2      : on stderr
```

Read it against §4. The first argument names the program. The field `args` is the argument vector the program will receive. The word `piped` says that slot 1 and slot 2 are to be handed back rather than joined to the terminal. The call `cmd.output` does the `fork` and the `exec`, waits for the copy to finish, and returns three things. The exit status, and the bytes written to each of the two output slots.

Two details matter for what follows. The bytes come back as bytes, so a `TextDecoder` turns them into a string. A program that writes a line writes a newline at the end of it, and that newline is in those bytes, so the method `trim` is used to take it off.

```
const cmd = new Deno.Command("sh", { args: ["-c", "echo hello"], stdout: "piped" })
;
const { stdout } = await cmd.output();
const text = new TextDecoder().decode(stdout);

console.log("as decoded :", JSON.stringify(text));
console.log("trimmed    :", JSON.stringify(text.trim()));
```

```
$ deno run --allow-run tr.ts
as decoded : "hello\n"
trimmed    : "hello"
```

The method `trim` removes whitespace from both ends of a string. It is the whole of what it does. And the call is awaited, which is what makes a spawned process usable from a server. The whole descent is a suspension, and the flow of control goes back to the loop while the child runs.

12.1 Deno can call Claude Code

Claude himself is a large language model running on Anthropic's servers. The Claude Code harness is something else entirely. It is a TypeScript program running on Unix as a CLI tool, and it is the only part of the arrangement we can point at.

So a web server of the kind in §11 can call it, because calling a CLI tool is exactly what the previous listing did. Here is a whole server that does this.

```
async function askClaude(prompt: string): Promise<string> {
  const cmd = new Deno.Command("claude", {
    args: ["-p", prompt],
    stdout: "piped",
  });
  const { code, stdout } = await cmd.output();
  if (code !== 0) throw new Error(`claude exited ${code}`);
  return new TextDecoder().decode(stdout).trim();
}

Deno.serve({ port: 8097 }, async (req) => {
  const body = await req.json();
  if (typeof body.ask !== "string") return json({ error: "expected {ask: string}" }, 400);
  return json({ reply: await askClaude(body.ask) }, 200);
});
```

And here is how we make a request to that server.

```
$ curl -s -X POST http://127.0.0.1:8097/ -d '{"ask":"Reply with exactly the
word: yes"}'
{"reply":"yes"}
```

A prompt arrived over a socket. It went down into a process. A response came back up and was answered with. That is the whole of the game, at one level.

13 Delegation

The function `delegate` carries one prompt to one port and then spawns a new process. The output of this new process is the response.

The process it spawns runs `delegate.ts`, which is a small CLI tool of the game's own. That tool receives a port, a prompt and a depth as its argument vector. It makes one HTTP request to the server listening on that port. It writes the response on standard output and nothing else, so the caller can parse the whole of what comes back. Its own comment says that this file is the mapping *u*, made executable.

```
export async function delegate(  
  port: number, prompt: unknown, depth: number,  
) : Promise<unknown> {  
  const cmd = new Deno.Command("deno", {  
    args: ["run", "--quiet", "--allow-net", "delegate.ts",  
          String(port), JSON.stringify(prompt), String(depth)],  
    stdout: "piped",  
  });  
  const { code, stdout } = await cmd.output();  
  const text = new TextDecoder().decode(stdout);  
  if (code !== 0) throw new Error(`delegate to :${port} exited ${code}`);  
  return JSON.parse(text) as unknown;  
}
```

The prompt goes down as an argument. The response comes back on standard output. So one hop costs one process, which is deliberate. The boundary is a real one rather than a function call wearing a costume.

The key point of all of this is that Claude is just another CLI tool, and thus just another container. Our example game `stalin` does not yet include a typed agent, but all the machinery is here to add one. Perhaps a corrupt commissariat who has some ideas of his own about how to run the country.

14 A container in TypeScript

A container is a pair. The prompts it accepts, and for each prompt the type of its response. Writing the second half down is the difficulty, because a response type depends on a value.

TypeScript has no dependent pairs. It does not need them here, and the reason is that the prompts are finite. Over a finite index a dependent sum is a union.

$$\Sigma_{s:S} B\ s = \bigcup_{s \in S} \{s\} \times B\ s$$

So a container is presented not as a pair of types, but as the union of its pairs, one pair for each prompt. The game calls one such pair a *fibre*.

```
export interface Fib<S, P> {  
  readonly shape: S;    // one prompt  
  readonly pos: P;      // the response to that prompt  
}
```

No value of that type is ever built. It exists to hold two types together. A container is a union of them, and here is one with two prompts.

```
type ShelfC =  
  | Fib<{ tag: "Book"; id: number }, Book>  
  | Fib<{ tag: "Count" }, number>;
```

Both halves are now written down. What remains is to read them back out, and that is what the two names in the next section do.

```
export type Shape<C> = C extends Fib<infer S, unknown> ? S : never;  
  
export type PosAt<C, S> =  
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

The keyword `infer` names a type the checker is to work out for itself. So the first definition says this. If `C` is a fibre over some prompt type, call that type `S` and give it back. A conditional type distributes over a union, so asking it of the whole union gives the union of the prompts.

The second definition asks two questions rather than one. Is `C` a fibre? If so, call its prompt type `S1` and its response type `P`. Then, does the prompt `S` you are holding lie in that fibre? If it does, give back `P`. If it does not, give back `never`.

Distributing over the union asks that of every fibre. All but one answer `never`, and `never` contributes nothing to a union, so exactly one response type survives. That is the dependent lookup, and it is exact because the prompts are disjoint.

The direction of the second test is worth noticing. It is `S extends S1` and not the other way round. A prompt you actually hold is usually narrower than its fibre's declared prompt type, because it carries a literal tag and particular fields. What you want is the fibre it lies in.

```
Shape<ShelfC>  
PosAt<ShelfC, { tag: "Count" }>  
PosAt<ShelfC, { tag: "Book"; id: number }>
```

The first is the two prompts, which is a tagged union in the sense of §8. The second is **number**. The third is **Book**. Those two are conditional types in the sense of §9, and each keeps the one fibre whose prompt it was asked about.

That pair of definitions is the “fake” the introduction promised. The dependency is real, in that a response type is computed from a prompt. It is bought with a union and a conditional type rather than with a dependent sum, and it works because the prompts are finite and carry tags.

15 Runners

A *runner* is a container you can send a prompt to.

So far a container has been a pair of types. To actually send a prompt you need one thing more, namely where the thing that will answer is listening. A runner is a container together with the means of reaching it.

```
export interface Runner<C> {  
  readonly label: string;  
  run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>>;  
}
```

The field `label` is a name for the container. It is what a failed validation reports, and it turns out to do more than that, which §19 comes back to.

Both names in that type are the ones §14 defined.

Now read the type of `run`. It takes a prompt, which must be one the container accepts. It gives back a promise of the response that this container gives to that prompt. So the second half of a container, meaning for each prompt the type of its response, is written down here and checked by the compiler.

15.1 Decoders

A *decoder* takes an `unknown` and returns either a value of the type expected or `null`. It is the validate rather than cast discipline of §10, applied at the one place where untyped bytes enter the program.

A runner needs one decoder for each prompt it accepts, so the decoders come as a table. Take a shelf that answers two prompts. A `Book` prompt is answered with a book, and a `Count` prompt with a number.

```
const asBook = (u: unknown): Book | null => {  
  if (!isRecord(u)) return null;  
  if (typeof u.id !== "number") return null;  
  if (typeof u.title !== "string") return null;  
  return { id: u.id, title: u.title };  
};  
  
const asCount = (u: unknown): number | null =>  
  typeof u === "number" ? u : null;  
  
const decoders = { Book: asBook, Count: asCount };
```

That is the whole of it. What arrives is `unknown`. The decoder for that prompt says whether it is the response it ought to be, and only then is it treated as one.

15.2 Pass-through runners

A *pass-through* runner uses none of the combinator objects. It serves one container on one port. It passes the prompt through to the server there, and passes the response back. The game's constructor for one of these is called `leaf`.

```

export function leaf<C>(label: string, port: number, decoders: Decoders<C>): Runner
  <C> {
  return {
    label,
    async run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>> {
      const raw = await delegate(port, shape, depth); // unknown, off the wire
      const value = decoders[shape.tag](raw); // the decoder for this
      prompt
      if (value === null) throw new Error(`${label}: reply did not validate`);
      return value as PosAt<C, S>;
    },
  };
}

```

The interface in §15 gave `run` a type and no body. Here is the body. The type parameters are the same bookkeeping as before, and what they surround is three lines of work.

The prompt descends, by `delegate`. What comes back is `unknown`, because JSON off a wire carries no type. The tag of the prompt chooses a decoder, and that decoder either returns a response or returns `null`. Nothing is cast until it has been checked, and a response that does not validate throws an error rather than travelling further up.

The argument `decoders` is a table with one entry for each prompt. Its type is a mapped type over the tags, in the sense of §9.9, and it buys exactly two guarantees.

- Every prompt has a decoder. A missing one does not compile.
- Each decoder returns that prompt's own response type, and no other.

Those two together are what a dependent function would have given for free.

And notice what became of the response type. It never crossed the wire. Only the prompt went down. The response type was recovered on this side, by the runner, which knew which prompt it had sent.

15.3 And this is the morphism

Put the pieces together and the destination arrives in two lines. Here again is the definition from the introduction. A *morphism* of containers $(P, R) \rightarrow (Q, T)$ is a pair of functions

$$u : P \rightarrow Q \qquad f : (p : P) \rightarrow T(u\,p) \rightarrow R\,p.$$

The mapping u takes a prompt of the upper container to a prompt of the lower one. The function f takes a response to that lower prompt, and a record of which upper prompt was sent, and gives a response to the upper prompt. So u goes down and f comes back.

One name is needed to write that down. The name `lower` is a runner for the lower container, the one written (Q, T) above. A pass-through runner will do, and then `lower.run` is one hop to the server that serves it.

```

const reply = await lower.run(u(prompt)); // u: the prompt, going down
return f(prompt, reply); // f: the response, coming back

```

The mapping u runs on the way down. Then the function stops. The frame is set aside and the flow of control goes back to the loop. In this implementation the descent is a real one, because a process is spawned and a socket is opened. Control genuinely leaves this program. When the response arrives it resumes this frame at this line, and f runs on the way back up.

Control down, data up, in one function body, read top to bottom, with the suspension invisible in the text. That is what the algebra says a morphism is. It is one keyword doing all of it.

16 The tensor combinator

16.1 Concurrency is not parallelism

Awaiting two promises one after the other runs them one after the other. To have both in flight you must start both before awaiting either. The operator that does it is `Promise.all`. It takes a list of promises and produces a promise of the list, settling when *all* of them have.

```
const ask = (ms: number) => new Promise(r => setTimeout(() => r(ms), ms));

let t = performance.now();
await Promise.all([ask(100), ask(100)]);
console.log("both in flight", Math.round(performance.now() - t), "ms");

t = performance.now();
await ask(100);
await ask(100);
console.log("one after the other", Math.round(performance.now() - t), "ms");
```

The call `performance.now` reads a clock, in milliseconds, so the difference between two readings is how long the lines between them took.

```
$ deno run d3.ts
both in flight      103 ms
one after the other 204 ms
```

Nothing ran in parallel in either line. There is one flow of control. What differs is whether the two waits overlap. This is also the first point at which two prompts exist at the same time, which is what makes a combinator possible at all.

16.2 The tensor combinator

Here again is the line that put both prompts in flight.

```
await Promise.all([ask(100), ask(100)]);
```

Read it with the container algebra in front of you. Both sides are prompted. *Both responses are owed*. If either rejects, the whole thing rejects. There is no outcome in which one response arrives and the other is excused. That is the tensor, exactly.

Now the game's `delegateAll` is one line, and its comment says so.

```
/** Fire several prompts at once and require a reply from each -- the tensor's
    defining move. */
export const delegateAll = <T extends readonly [number, unknown] []>(
  calls: T, depth: number,
): Promise<unknown []> =>
  Promise.all(calls.map(([port, prompt]) => delegate(port, prompt, depth)));
```

That line needs unpacking.

- The type `[number, unknown]` is a pair. A port number, and a prompt whose type is not yet known.
- So the argument `calls` is a list of such pairs.
- The word `readonly` says this function will not alter the list it was handed.
- The method `calls.map` applies a function to every element and collects the results. Here every pair becomes one promise.
- The pattern `([port, prompt])` takes a pair apart in the place where the parameter would be named.
- The function `delegate` fires one prompt at one port and returns a promise of the response. That is the descent, and it spawns a process.
- The operator `Promise.all` turns the list of promises into one promise of a list.
- The number `depth` is how far down the tower this hop is. It rides along so the trace can indent, and it plays no part in the algebra.

So a list of pairs goes in, one port and one prompt in each, and one promise of a list of responses comes out. Notice that nothing is awaited here. The caller decides when to wait, and until it does, every prompt is in flight.

17 The other three combinators

The other three fall out of the same reading.

combinator	written as	because
$\boxtimes$ tensor	<code>await Promise.all(...)</code>	all are prompted, all responses owed
$\times$ product	<code>await</code> one of them, chosen	both offered, exactly one answers
$\triangleleft$ sequential	<code>await a</code> , then <code>await b(response)</code>	the second prompt is computed from the first response
$+$ sum	a <code>switch</code> on the tag	one side or the other

The sequential row is only expressible because `await` returns a value. The second prompt is computed from the first response.

The product row merits a remark. Nothing in `Promise` enforces that exactly one of the two responses is used. An un-disciplined programmer could use both responses and mess everything up.

18 Formal verifiability

The short answer is that this design cannot be formally verified. The algebra motivates the architecture. It does not verify it.

The algebra is a design vocabulary, and it was used as one.

The algebra is the reason the game is five processes rather than one. A morphism has a level above and a level below, and making the boundary a socket makes that claim falsifiable.

The algebra is the reason each turn is a named composite rather than a block of code. The algebra offers four ways to combine, and forces the designer to say which.

And it is the reason the central mechanic of the game is what it is. The combinators $\boxtimes$ and $\times$ have identical shapes and differ only in obligation. So choosing between them is choosing between doing two things and choosing between two things. In a game about allocating one stock of steel, that *is* the design decision. Had that choice been made without the vocabulary to name it, it would have been made worse.

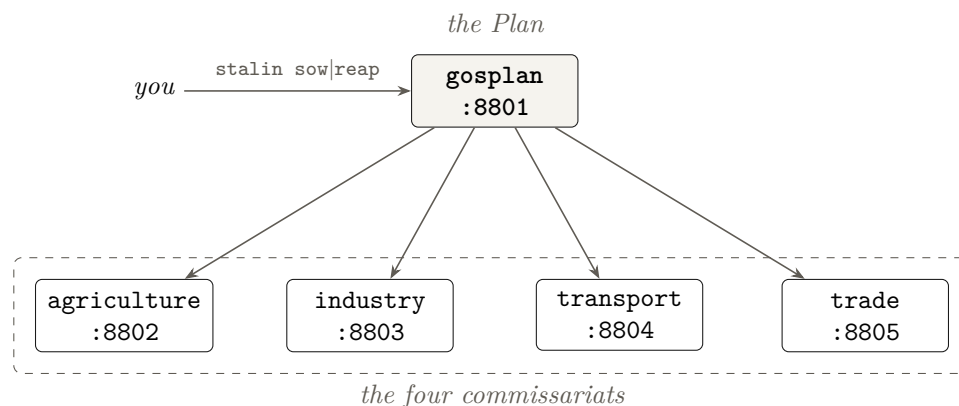
So here is the honest statement of what the algebra bought. Better decomposition. A forced decision at every junction. And a name for the distinction the game turns on. Not a theorem. The formalism is load-bearing for the *design* and carries no weight in the *verification*. We make no claim otherwise.

This is the take-home point of the entire note. What is built below is a fake. It is a container algebra impersonated in a language with no dependent types. The temptation is to read that as a confession of laziness, repairable by choosing a better language. It is not. The faking is forced by IO.

19 How the game is implemented

19.1 Five servers, and why they are servers

The game runs as five programs, each listening on a port. The player talks only to the Plan. The Plan delegates to the four commissariats beneath it.



Every arrow is a delegation, and every arrow points away from the Plan. The Plan is the only client any of them has. The four do not talk to each other.

19.2 The game as a composite

Each turn is one of the four compositions. Which one it is makes a claim about the year, rather than being a detail of the code.

	composite	why that one
spring	(agri $\boxtimes$ mill) $\boxtimes$ rail	all three work at once and <i>all three owe a report</i> . There is no partial year
autumn	transport $\triangleleft$ trade	what may be sold is computed <i>from</i> what the railway actually moved
build	tractors $\times$ armaments	choose to invest in either tractors or armaments

19.3 What crosses, and what the status code says

Every hop is a POST to a port with a JSON body, and the response is JSON. The port identifies the container. The three outcomes are genuinely different kinds of thing, and the status vocabulary already distinguishes them.

```
try { body = await req.json(); }
catch { return json({ error: "not JSON" }, 400); }

const prompt = opts.parse(body);
if (prompt === null) {
  return json({ error: "not a valid prompt for this container", got: body }, 400);
}
try { return json(await opts.answer(prompt, depth + 1), 200); }
catch (e) { return json({ error: msg }, 500); }
```

The depth rides in a header rather than in the prompt, because the prompt is the container's shape, and nothing that isn't a prompt belongs in it.

19.4 A hop is a CLI tool

The program that carries a prompt to the next level is fifty lines long, and knows nothing about the game. With the agriculture commissariat running, a single prompt can be fired at it by hand.

```
$ deno run --quiet --allow-net --allow-env lib/delegate.ts 8802 \  
  '{"tag":"Harvest","workforce":{"fields":100,"drivers":0,"mill":12,  
    "rail":8,"dead":0},"tractors":0,"year":1928}' 0  
  '{"grain":263.35,"grade":"adequate","perWorker":2.63,"withoutTractors":263.35}'
```

That is the mapping u made executable, and it observes every discipline from §4. Its exit codes distinguish the two kinds of failure. Exit code 2 for a malformed prompt or missing arguments. Exit code 1 for a port with nothing behind it. Every diagnostic goes to descriptor 2, and descriptor 1 carries the response and nothing else. That is exactly what lets the caller parse the whole of stdout as one JSON document.

That discipline is also what makes the tracing possible. The servers narrate themselves on descriptor 2 while descriptor 1 stays machine-readable.

```
↓ gosplan  Sow order={"procurement":"firm","labour...":{"  
  ↓ gosplan  ((agri (x) smelter) (x) transport)  
    ↓ agri    Harvest workforce={"fields":96,"drivers":0,"mill...":1 tractors=0  
      year=1928  
        ↑ agri    {grain, grade, perWorker, withoutTractors}  
          ↓ industry Smelt workers=14 millCapacity=8 year=1928  
            ↑ industry {steel, idleCapacity}  
              ↓ transport Lay workers=10 steel=10 year=1928  
                ↑ transport {added, steelUsed}  
          ↑ gosplan  {kind, year, harvest, steel, ...}
```

Indentation is the depth counter. The arrow ↓ is control flowing down. The arrow ↑ is data flowing back up. The tracer prints at most thirty-four characters of any value and then gives up, which is what the ... is.

The second line is the one to look at. It is a runner's `label`, and labels compose. A pass-through runner is given one when it is built. Every combinator makes its own out of the labels of its parts, in the algebra's own notation.

```
label: `(${a.label} (x) ${b.label})`    // the tensor  
label: `(${a.label} x ${b.label})`      // the product  
label: `(${a.label} <| ${b.label})`     // the sequential  
label: `(${a.label} + ${b.label})`      // the sum
```

So `((agri (x) smelter) (x) transport)` is not a comment anybody wrote. It is the expression that built that runner, printing itself, and the four hops indented beneath it are what the expression turned into. That is a derivation tree of container morphisms, printed by a program that had no idea it was doing that.

19.5 Where the compiler carries a rule of the game

Two rules are enforced by the type checker rather than validated at run time. Both work by the trick from §7.1, a conditional type over a finite union.

```

export type ExportChoice<G extends HarvestGrade> =
  G extends "failure" ? "none"
: G extends "poor" ? "none" | "surplus"
: G extends "adequate" ? "none" | "surplus" | "full"
: "none" | "surplus" | "full" | "maximum";

exportPlan("bumper", "maximum") // fine
exportPlan("failure", "maximum") // does not compile

```

Exporting grain during a failed harvest is not a move the engine rejects. It is a move that cannot be written down. The same trick carries the arc of the whole game. You may buy steel while in deficit, and never sell it. A country that exports its steel is not industrialising.

```

export type SteelTrade<P extends SteelPosition> =
  P extends "deficit" ? "none" | "buy" : "none";

```

Both work only because the index set is finite. A rule over tonnages could not be typed this way. The same rule over *grades* can. That is also how the apparatus actually worked. Quotas were set against reported categories, never against measured tonnes.

19.6 The one thing that cannot cross

Of everything JSON leaves out, one omission bites structurally rather than merely requiring care. A function cannot be serialised. And the shape of $\triangleleft$ is a first prompt together with a function computing the second.

So sequential composition cannot be distributed over the wire. It is resolved *above* it, in the runner. The runner prompts the first container, then applies that function to the response to obtain the second prompt. Only the two ground prompts ever cross.

The wire format is not neutral about which combinators can be sent and which must be interpreted locally. This is the sharpest example in the repository of a mathematical structure meeting a transport and losing an argument.

19.7 What the reports are worth

Each commissariat holds two things the Plan cannot see. How well it is actually doing. And how much it inflates its dispatches. The Plan keeps the true ledger for grain, because the grain physically exists. What diverges is the *report*. And you plan against the report.

```

Narkomtiazhprom: quota fulfilled, with difficulties overcome.

```

This is the honest limit of everything above. The type of a dispatch guarantees its shape and says exactly nothing about its truth. A validator can establish that a response is a well-formed dispatch. No validator can establish that the number in it is the number that happened. The reckoning prints both columns and leaves the reader to notice.

19.8 And where the protocol is not doing the work

One last piece of honesty. HTTP is stateless and these servers are not. Each keeps its books at module scope, in the process. That is defensible. A commissariat *is* a stateful thing, and the exercise is about the container algebra rather than about session management. But it has

a consequence, and the consequence is visible in the interface. The protocol cannot clear that state. So clearing it has to become a prompt.

```
Reset: (p) => { seed = p.seed; books = new Books(...); laid = 0; lastQuota = 0;
              return { ok: true }; },
```

The prompt `Reset` has no meaning in the fiction. No plan resets a railway. It exists because starting a new game must reach the commissariats. Otherwise the weather and the books carry over into the next one, and the comment records that as a bug that actually happened. Determinism on the seed is what makes a playthrough a regression test, so this is load-bearing rather than tidying. And the general lesson is the one worth taking away. When process state substitutes for protocol state, the lifecycle leaks into the interface.

Conclusion

What has been built is a *fake*. Containers for event-driven programming, faked with CLI tools and TypeScript `async` event handlers.

Ghani's combinators are a construction in dependent type theory. TypeScript has no dependent types, and Unix has no types, and nothing here added either. What the game does instead is impersonate the construction with two ordinary things, one supplying each leg of a morphism: command-line tools carry the prompt down, and TypeScript `async` event handlers carry the response back up.

The impersonation has three parts. Each one has appeared above.

A container (P, R) becomes a tagged union together with a lookup indexed by its tags. That works, and works only, because the tag set is *finite*. Over a finite index a dependent type is just a table, and a table is something TypeScript can write down.

The inhabitant $(p : P) \rightarrow R p$ becomes the handler record. The compiler enforces its totality, because it is a mapped type over that same union.

And a morphism becomes an `async` function. That is where the two directions meet.

```
const reply = await lower.run(u(prompt)); // the prompt, going down
return f(prompt, reply);                  // the reply, coming back
```

19.9 The response flows backwards

That is the part worth dwelling on. Control flows *down*. A handler is woken by a prompt. It computes a lower-level prompt with u , and fires it. In this implementation that means spawning a process and opening a socket, so the descent is a real one, made of real programs. Then it stops. At the `await` the handler suspends and hands the runtime back. The runtime is free to wake other handlers while it waits.

The response then flows *backwards* along the path the prompt took. Each response resumes the handler that fired the prompt, at the very line it stopped on. That handler applies f to turn the response into its own. So u runs on the way down and f on the way back up, in one function body, read top to bottom, with the suspension between them invisible in the text.

The arrows in the trace are exactly this. The arrow $\downarrow$ is a prompt descending as a process. The arrow $\uparrow$ is a response climbing back as a resumed function. It is also why a server here

can be a client without ceasing to be a server. The Plan sits mid-response with four prompts outstanding beneath it, still listening, because suspending is not the same as being busy.

And where the fake fails, it fails visibly. Sequential composition cannot be sent over the wire, because its shape contains a function and functions do not serialise. A validated response is guaranteed to have the right shape, and is guaranteed nothing about its truth. Both limits are consequences of the same substrate that made the impersonation possible in the first place. That is the honest summary of the whole exercise. The algebra is real, and everything holding it up is Unix.